NAME

hc_malloc – Sets up and manages memory allocation arena.

DESCRIPTION

hc_malloc is a library written in C that sets up an information structure that keeps track of heap memory allocations. It is a wrapper around the standard memory allocation/manipulation functions and keeps track of the addresses.

Installation scripts work seamlessly on Unix, Linux and macOS. *Not tested on MS Windows(tm).*

SYNOPSIS

hc_init()

This function should be called fairly early in your program. It will set up an array that can contain up to 32 address pointer that can automatically be expanded in 32 additional pointer locations at the time when needed. Capacity and next available index values are part of this structure.

Arguments: *none*
Return Value: *int* [0 (succes) or 1 (fail)]

hc_cleanup()

You should call this functions at the end of your process. It will free all addresses in the address array. The memory allocated for the arena structure will be freed as well.

Arguments: *none*
Return Value: *none*

hc_free(addr_pntr)

Will free the memory allocated at the specified address. The address will be removed from the array.

Arguments: *void** [address to be released]
Return Value: *none*

hc_display()

Displays the content of the arena structure, including the address array.
Intended for light debugging.

Arguments: *none*
Return Value: *none*

hc_malloc(size)

Allocates given size memory and add it to the array and updates the next index value.

Arguments: *size_t* [size in bytes]
Return Value: *void** [address pointer]

hc_calloc(num_elem,elem_size)

Allocates requested memory block. Size is calculated by multiplying the number of element requested by the size of the element. Address will be added to the arena array and updates the next index value.

Arguments: *size_t* , *size_t* [number of elements, size of element]
Return Value: *void** [address pointer]

hc_posix_memalign(memptr,alignment,**size)**

Allocates size bytes of memory such that the allocation's base address is an exact multiple of the system's page size alignment, and returns the allocation in the value pointed to by given *memptr*. Use this when writing standard software for Linux, macOS or UNIX. Every modern Linux, macOS and BSD system fully supports this.

Arguments: *void** , *size_t* , *size_t* [address pointer, alignment, size allocation]
Return Value: *int* [0 (success), !0 (failure)]

hc_realloc(old_pntr,size)
Reallocates the existing memory alloction pointed to by the old_pntr argument to a new area with the specified size. The original address in the array will be updated with the new one.

Arguments: *void** , *size_t* [address pointer, new size allocation]
Return Value: *void** [address pointer]

hc_reallocf(old_pntr,size)
Reallocates the existing memory alloction pointed to by the old_pntr argument to a new area with the specified size. The original address in the array will be updated with the new one. This is a drop-in replacement to the standard **reallocf** function. The **hc_realloc** function already took care of the release of the old pointer.

Arguments: *void** , *size_t* [address pointer, new size allocation]
Return Value: *void** [address pointer]

hc_strdup(addr_pntr)
Copies the content of a string pointed to by the addr_pntr. Updates the address array and next_index value.

Arguments: *void** [address pointer to string to be duplicated]
Return Value: *char** [address pointer]

INTERNAL FUCTIONS

_hc_address_array_capacity_increase()
This function will be invoked when the current capacity of the address array is reached. The increments is by 32 address locations each time.

Arguments: *none*
Return Value: *none*

_hc_address_add(addr_pntr)
Adds an address pointer to the address array. Will evoke _hc_address_array_capacity_increase() when the current capacity is reached

Arguments: *void** [address pointer to be added]
Return Value: *none*

_hc_address_locate(addr_pntr)
Returns the index of the given address pointer.

Arguments: *void** [address pointer to locate]
Return Value: *int* [index to address array]

_hc_address_replace(old_pntr,new_pntr)
Locates the given old pointer in the address array and replaces it with the new given pointer.

Arguments: *void** , *void** [old address pointer, new address pointer]
Return Value: *none*

EXAMPLE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <hc_malloc/hc_malloc.h>

#define LOG_ERROR(msg) fprintf(stderr, "[ERROR] in %s: %s\n", __func__, msg)

int main() {
      /* initializes arena structure */
      if ( hc_init() ) { return 1; }

      /* string pointer on the stack -> .rodata (text-) segment */
      char *a_string_1 = "Now the time hase come for all good men to come to the aid o

      /* Allocates a 80 character sized area on the heap */
      char *a_string_2 = hc_malloc(80 * sizeof(char));
      if( a_string_2 == NULL ) { LOG_ERROR("hc_malloc"); return 1; }

      /* Moves some content to the heap allocated string */
      strcpy(a_string_2, "Look, a string on the heap!");

      /* Duplicates the stack pointed string to the heap */
      char *a_string_3 = hc_strdup(a_string_1);
      if( a_string_3 == NULL)  { LOG_ERROR("hc_strdup"); return 1; }

      /* Displays the address pointers and other arena values */
      hc_display();

      /* frees all the allocated memory and removes the arena structure */
      hc_cleanup();

      return 0;
}
```

TEST

HOME/.local/share/hc_malloc/tests/test_hc_malloc

This test facility will loop through a number scenarios, displaying the state of the Heap Control arena values. It will demonstrate allocating and removing allocated data areas and their content. Heap arena address array will be triggered to expand. An example of page boundry alignment allocation will be proven and a total reset of the arena data array and associated control values.

FILES

/usr/local/share/hc_malloc/doc/README.md
: Documentation to this utility, also available in html and pdf form.

https://github.com/yveshoebeke/hc_malloc
: The README.md contains installation instructions.

LICENSE

/usr/local/share//hc_malloc/doc/LICENSE
: MIT license.

AUTHOR

yves hoebeke (bytesupply@proton.me)

SEE ALSO

malloc(3), **leaks**(1), **free**(3), **posix_memalign**(3), **realloc**(3), **sysconf**(3)